

Proyecto: Huellas Vivas

Plataforma web para la gestión y adopción de mascotas

Ingeniería Civil en Informática · Base de Datos

Máximo Torrijo · Joaquín Molina · Ignacio Jorquera

1. Introducción y contexto del problema

Todos hemos visto mascotas deambulando por calles, plazas y terminales. Detrás de esto hay varias causas conocidas. Mucha gente adopta o compra una mascota por impulso y después no puede o no quiere mantenerla: por el gasto en veterinario y comida, por una mudanza, o porque el animal crece y da más trabajo de lo que pensaban. A eso se suma que muchos animales no están esterilizados, así que las camadas no planificadas terminan en la calle. El resultado es que hay animales que pasan hambre, se enferman, sufren accidentes o se reproducen sin control, y cada año son más los que se quedan sin hogar.

Frente a esto, los refugios y las agrupaciones de rescate independiente rescatan animales, los alimentan, los llevan al veterinario, los vacunan y los mantienen en albergues o en casas de acogida temporal. Su meta es que cada animal encuentre un hogar definitivo. Muchas de estas agrupaciones funcionan con voluntarios y con pocos recursos, así que su tiempo es limitado.

El rescate funciona, pero el último paso, la adopción, es donde todo se complica. Estas agrupaciones no tienen una herramienta propia para organizarse y suelen usar cuadernos, planillas de Excel y chats de WhatsApp o Instagram. Cuando alguien quiere adoptar, la información queda repartida en muchos lugares, las postulaciones se pierden entre cientos de mensajes y los voluntarios responden una y otra vez las mismas preguntas. A veces ni siquiera está claro si un animal sigue disponible o ya fue adoptado, y las respuestas pueden tardar semanas.

Problema: no existe un lugar único donde consultar la información. Esto genera desorden en las adopciones, postulaciones que se pierden en los chats y confusión sobre si un animal sigue disponible.

Afectados: los animales son los más perjudicados, porque cada semana de demora es una semana más en un albergue o en un hogar temporal, y además ese espacio no se puede usar para rescatar a otro. Los refugios y voluntarios pierden tiempo en tareas repetitivas que podrían dedicar a cuidar animales. Y las personas que quieren adoptar se frustran por la falta de información y la lentitud, y algunas terminan desistiendo.

Objetivo de negocio: lograr que el proceso de adopción sea ordenado, claro y rápido, con un solo sistema donde toda la información esté al día.

Propuesta de solución: una página web con dos partes. Una pública, donde cualquiera puede ver las fotos y fichas de los animales disponibles y postular fácilmente. La otra es un panel privado donde el refugio revisa las solicitudes, las aprueba o rechaza y mantiene el catálogo al día.

1.1 Justificación de la solución propuesta

Elegimos una página web porque se abre desde cualquier celular o computador sin instalar nada, y porque el enlace de cada animal se puede compartir en redes sociales, lo que ayuda a que más gente lo conozca. Descartamos WhatsApp y Excel porque dependen de que una persona no se equivoque ni se le olvide algo. En la

página, en cambio, cuando una mascota es adoptada el sistema la saca de la lista de inmediato y queda un registro claro de cada adopción.

1.2 Tecnologías utilizadas

Para construir el sistema usamos las siguientes herramientas. Junto a cada una explicamos qué hace y para qué la necesitamos:

- **Node.js (el servidor):** es el entorno que ejecuta el backend con JavaScript: recibe lo que pide la persona, lo procesa y le responde. Puede atender a muchos usuarios al mismo tiempo (RNF07).
- **Express.js (rutas y capas):** define qué pasa en cada endpoint de la API y separa el código en rutas y controladores (RNF08), enviando los datos en formato JSON (RNF06).
- **MySQL (la base de datos):** guarda usuarios, mascotas y solicitudes en tablas relacionadas y permite transacciones: los cambios de una adopción se guardan juntos o ninguno (RNF02).
- **mysql2 (mysql2/promise):** librería que conecta Node.js con MySQL mediante un pool de conexiones y consultas con parámetros.
- **Swagger UI (swagger-ui-express y swagger-jsdoc):** documenta la API y permite probar cada endpoint directamente desde el navegador, en la dirección /api-docs.
- **dotenv (variables de entorno):** guarda en el archivo .env los datos de conexión a MySQL, para que las credenciales no queden escritas en el código.
- **CORS:** permite que la futura interfaz web pueda hacer peticiones a la API desde otro origen.

2. Requerimientos

2.1 Requerimientos funcionales (RF) y no funcionales (RNF)

Requerimientos funcionales

RF01 – Registro e inicio de sesión – Cualquier persona puede crear una cuenta y entrar con su correo y contraseña. El sistema reconoce si quien entra es un usuario que busca adoptar o un administrador del refugio, y le muestra solo lo que le corresponde.

RF02 – Catálogo de mascotas disponibles – Muestra una lista visual con la foto principal, los datos básicos y el estado de los animales que buscan hogar. Los animales que ya fueron adoptados desaparecen automáticamente para no generar confusión.

RF03 – Búsqueda y filtros – El visitante puede buscar combinando 2 filtros obligatorios (Estado, que debe ser 'Disponible', y Ciudad) con 4 filtros opcionales (Especie, Tamaño, Sexo y si está Esterilizado). Así, una persona encuentra rápido el animal que mejor calza con su hogar; por ejemplo, solo gatas esterilizadas en Valparaíso.

RF04 – Enviar solicitud de adopción – Quien tenga la sesión iniciada puede apretar un botón en la ficha de un animal para postular. La solicitud llega de inmediato al refugio, de forma ordenada.

RF05 – Mantenedor de mascotas – El administrador tiene un panel exclusivo para ver la lista completa de animales, agregar fichas nuevas con sus fotos, editar datos (como vacunas o descripción) y dar de baja o cambiar el estado de una mascota cuando haga falta.

RF06 – Mantenedor de usuarios – El administrador puede ver la lista de personas registradas, editar sus datos de contacto y suspender cuentas si detecta un mal uso, como postulaciones falsas. La cuenta no se borra, solo se desactiva (desactivación lógica).

RF07 – Aprobación y rechazo de postulaciones – El refugio revisa cada solicitud y decide si la aprueba o la rechaza. Al aprobar a un adoptante, el sistema cambia solo la mascota a 'Adoptada' y marca como 'Rechazada' cualquier otra solicitud pendiente para ese animal.

RF08 – Historial de postulaciones – Cada persona puede ver en su perfil las solicitudes que ha enviado y en qué estado están ('Pendiente', 'Aprobada' o 'Rechazada'), sin tener que preguntar por otros medios.

RF09 – Registro automático de fechas – El sistema anota solo la fecha de ingreso de cada animal y la fecha de su última modificación. Así el refugio sabe cuánto tiempo lleva cada mascota esperando un hogar y puede darle más difusión.

RF10 – Control de solicitudes repetidas – Si una persona ya postuló por un animal y esa solicitud sigue 'Pendiente', el sistema no le deja volver a postular por el mismo animal hasta que se resuelva la primera. Así se evitan duplicados y desorden.

Requerimientos no funcionales

RNF01 – Rapidez – El catálogo y la aplicación de filtros deben cargar en menos de 1,5 segundos, para que la navegación sea ágil y la gente no abandone la página.

RNF02 – Consistencia de los datos – La información de las adopciones nunca debe perderse ni quedar a medias. Si se aprueba una adopción, el cambio en la postulación y el cambio en la mascota se guardan juntos o no se guarda ninguno.

RNF03 – Protección de contraseñas – Las claves no se guardan tal como se escriben. Se transforman con bcrypt (con salt) para que nadie pueda verlas, ni siquiera los administradores del sistema.

RNF04 – Adaptación a pantallas (responsive) – El sitio se ve y funciona bien tanto en computador como en la pantalla pequeña de un celular.

RNF05 – Disponibilidad del catálogo – La parte pública con los animales debe estar disponible las 24 horas, para que cualquiera la revise cuando tenga tiempo.

RNF06 – Formato estándar de datos (JSON) – La información entre la página y el servidor viaja en formato JSON, un estándar de texto ordenado que evita errores y facilita hacer cambios a futuro.

RNF07 – Varios usuarios a la vez – El servidor debe poder atender a muchas personas al mismo tiempo sin ponerse lento ni caerse, por ejemplo si la publicación de un animal se hace viral en redes sociales.

RNF08 – Código ordenado por capas – El programa se separa en tres partes: lo que ve el usuario (Vista), la lógica que decide qué hacer (Controlador) y el acceso a los datos (Modelo). Así se puede cambiar el diseño sin rehacer el funcionamiento.

RNF09 – Protección de datos personales – La identificación del usuario y sus datos privados (como el RUT) se envían dentro de la petición y nunca aparecen en la dirección web (URL).

RNF10 – Respalos – El sistema permite hacer copias de seguridad periódicas de la base de datos, para recuperar la información si ocurre una falla.

2.2 Diagrama de casos de uso

En el sistema hay dos tipos de personas: el Usuario y el Administrador. El Usuario puede registrarse, iniciar sesión, consultar el catálogo, solicitar una adopción y revisar su historial. El Administrador puede iniciar sesión,

usar los mantenedores de mascotas y de usuarios, y administrar las solicitudes. Solicitar adopción, revisar historial, los dos mantenedores y administrar solicitudes necesitan que la persona tenga la sesión iniciada, por eso se conectan con Iniciar sesión mediante una relación «include» (para poder hacerlas, primero hay que pasar por Iniciar sesión). Consultar el catálogo, en cambio, es libre.

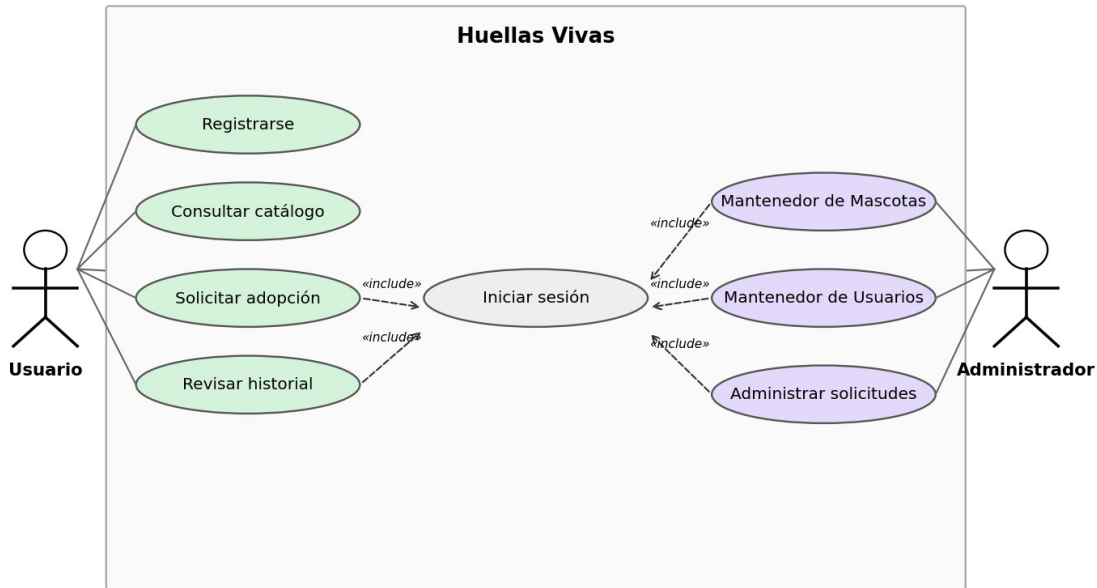


Figura 1: Diagrama de casos de uso

2.3 Descripción de casos de uso

Iniciar sesión

- **Quién participa:** Usuario y Administrador.
- **Precondición:** La persona ya debe estar registrada.
- **Flujo principal:** Escribe su correo y contraseña; el sistema revisa que sean correctos y reconoce si es Usuario o Administrador.
- **Postcondición:** La persona queda con la sesión activa según su rol.
- **Relación:** Es un paso obligatorio («include») antes de cualquier función que requiera sesión.

Registrarse

- **Quién participa:** Usuario.
- **Precondición:** El correo que se ingresa no puede estar usado por otra cuenta.
- **Flujo principal:** Llena un formulario con su RUT, nombre, correo, contraseña y teléfono; el sistema revisa que el RUT tenga formato correcto y que el correo no esté repetido.
- **Postcondición:** Se crea una cuenta nueva de Usuario, lista para usarse.
- **Relación:** Hay que registrarse antes de poder solicitar una adopción.

Consultar catálogo

- **Quién participa:** Usuario (o cualquier visitante).

- **Precondición:** No hace falta tener la sesión iniciada.
- **Flujo principal:** El sistema muestra las mascotas aplicando los 2 filtros obligatorios (Estado 'Disponible' y Ciudad) y permite aplicar de forma opcional otros 4 (Especie, Tamaño, Sexo y Esterilizado).
- **Postcondición:** Se muestran en pantalla las mascotas que cumplen con los filtros elegidos.
- **Relación:** No depende de ningún otro caso de uso.

Solicitar adopción

- **Quién participa:** Usuario.
- **Precondición:** Tener la sesión iniciada, que la mascota esté 'Disponible' y no tener ya otra solicitud pendiente para ella.
- **Flujo principal:** Entra a la ficha de la mascota y confirma que quiere postular; el sistema revisa que no exista una solicitud repetida (RF10) y guarda la postulación como 'Pendiente'.
- **Postcondición:** Queda registrada una solicitud 'Pendiente', ligada al usuario y a la mascota elegida.
- **Relación:** Necesita («include») haber iniciado sesión.

Mantenedor de mascotas

- **Quién participa:** Administrador.
- **Precondición:** Haber iniciado sesión como Administrador.
- **Flujo principal:** Consulta la lista de animales, agrega fichas nuevas con su foto, edita datos de salud o descripción, o da de baja una mascota cambiando su estado.
- **Postcondición:** Los datos o el estado de la mascota quedan actualizados, con la fecha de modificación registrada (RF09).
- **Relación:** Necesita («include») haber iniciado sesión.

Mantenedor de usuarios

- **Quién participa:** Administrador.
- **Precondición:** Haber iniciado sesión como Administrador.
- **Flujo principal:** Consulta la lista de usuarios, edita sus datos de contacto o suspende una cuenta (activo = 0) si detecta mal uso o postulaciones falsas.
- **Postcondición:** La información del usuario queda actualizada, o su cuenta queda desactivada (sin borrarla).
- **Relación:** Necesita («include») haber iniciado sesión.

Administrar solicitudes

- **Quién participa:** Administrador.
- **Precondición:** Debe haber al menos una solicitud pendiente por revisar.
- **Flujo principal:** Revisa una solicitud y decide aprobarla o rechazarla. Si la aprueba, la mascota pasa a 'Adoptada' y las demás solicitudes pendientes para ese animal se rechazan solas, todo dentro de una misma transacción (RF07).
- **Postcondición:** La solicitud queda 'Aprobada' o 'Rechazada'; si fue aprobada, la mascota queda 'Adoptada'.

- **Relación:** Necesita («include») haber iniciado sesión. A futuro podría ampliarse («extend») con un caso de uso de Agendar visita.

Revisar historial

- **Quién participa:** Usuario.
- **Precondición:** Tener la sesión iniciada.
- **Flujo principal:** El sistema busca todas las solicitudes que ha hecho esa persona y las muestra ordenadas por fecha, con su estado actual ('Pendiente', 'Aprobada' o 'Rechazada').
- **Postcondición:** Se muestra la lista de solicitudes hechas por el usuario.
- **Relación:** Necesita («include») haber iniciado sesión.

2.4 Diagramas de secuencia

Mostramos los dos procesos más importantes del sistema: filtrar el catálogo y postular a una adopción. En ambos se ve paso a paso cómo la persona interactúa con la pantalla (Vista), cómo la pantalla le pide información al Controlador y cómo el Controlador consulta o guarda datos en la Base de Datos, siguiendo el orden del patrón MVC del RNF08.

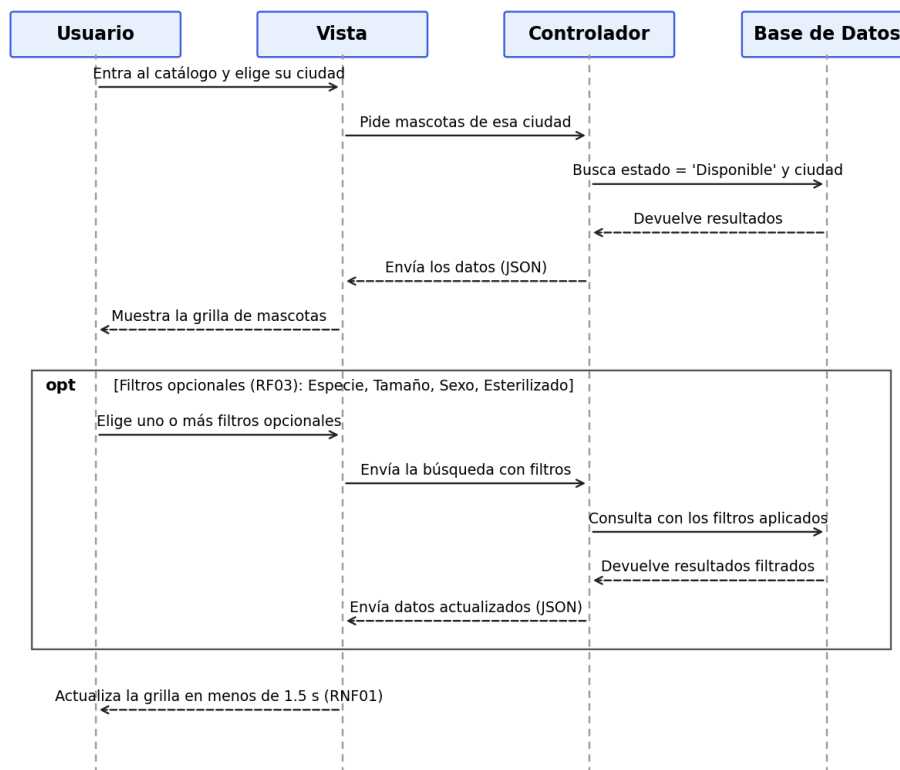


Figura 2: Diagrama de secuencia — Consulta y filtrado del catálogo

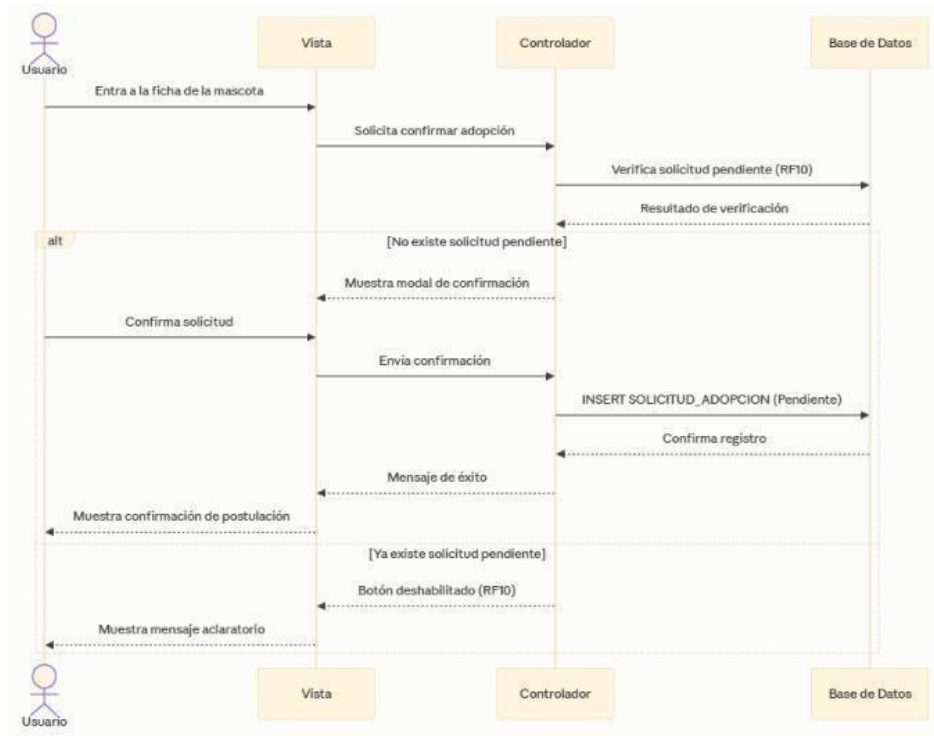


Figura 3: Diagrama de secuencia — Solicitud de adopción

3. Prototipo de los dos casos de uso

Al diseñar las pantallas buscamos que fueran simples y que a la persona le costara poco encontrar lo que necesita. Cada decisión de diseño que se explica a continuación responde a alguno de los requerimientos de la sección 2.1.

3.1 Caso de uso de consulta: catálogo y búsqueda

La pantalla tiene un panel de filtros a la izquierda y una grilla con las mascotas a la derecha. Lo ordenamos así porque solemos leer de izquierda a derecha y de arriba hacia abajo: primero se define qué se está buscando (los filtros) y después se revisan los resultados.

- **Relación con RF02 y RF03:** el catálogo muestra de entrada las mascotas 'Disponibles' de la ciudad elegida (los dos filtros obligatorios, Estado y Ciudad). Los selectores de Especie, Tamaño, Sexo y Esterilizado son opcionales y permiten afinar la búsqueda sin recargar la página.
- **Relación con RNF01 y RNF04:** los resultados se van cargando de a poco a medida que la persona baja en la página, para que la búsqueda no demore más de 1,5 segundos. Además, la pantalla está pensada primero para verse bien en el celular y después en el computador.

Huellas Vivas — Catálogo de mascotas

Filtros obligatorios

Estado

Ciudad

Filtros opcionales

Especie

Tamaño

Sexo: ☐ Macho ☐ Hembra ☐ Esterilizado

Foto

Firulais

Perro · Valparaíso · Macho

Disponible

Foto

Michi

Gato · Valparaíso · Hembra

Disponible

Foto

Rocky

Perro · Valparaíso · Macho

Disponible

Foto

Luna

Gato · Valparaíso · Hembra

Disponible

Figura 4: Prototipo — Catálogo y búsqueda

3.2 Caso de uso transaccional: ficha y solicitud de adopción

La pantalla de detalle de una mascota tiene un solo botón principal ('Solicitar adopción'), para que quede clarísimo cuál es la acción a seguir y no haya distracciones. Antes de guardar la solicitud aparece una ventana de confirmación que pregunta si la persona está segura, para evitar postulaciones enviadas sin querer.

- **Relación con RF04 y RF10:** si la persona ya tiene una solicitud en curso para esa mascota, el botón aparece deshabilitado, y así no se generan solicitudes repetidas.
- **Relación con RNF09:** el formulario envía el RUT y el correo del usuario de forma oculta al servidor, sin mostrarlos nunca en la dirección (URL) de la página.

Ficha de mascota

Firulais

Perro · Mediano · 2 años · Valparaíso

Juguetón y cariñoso, ideal para familias con espacio para jugar.

Solicitar adopción

Modal de confirmación

Confirmar solicitud

¿Deseas solicitar la adopción de Firulais?

Cancelar Confirmar

Figura 5: Prototipo — Ficha de mascota y solicitud de adopción

4. Modelo de datos

4.1 Modelo entidad-relación y modelo relacional

El sistema trabaja con cuatro entidades: ROL, USUARIO, MASCOTA y SOLICITUD_ADOPCION. En la Figura 6 se ven sus atributos (con la llave primaria subrayada) y cómo se relacionan.

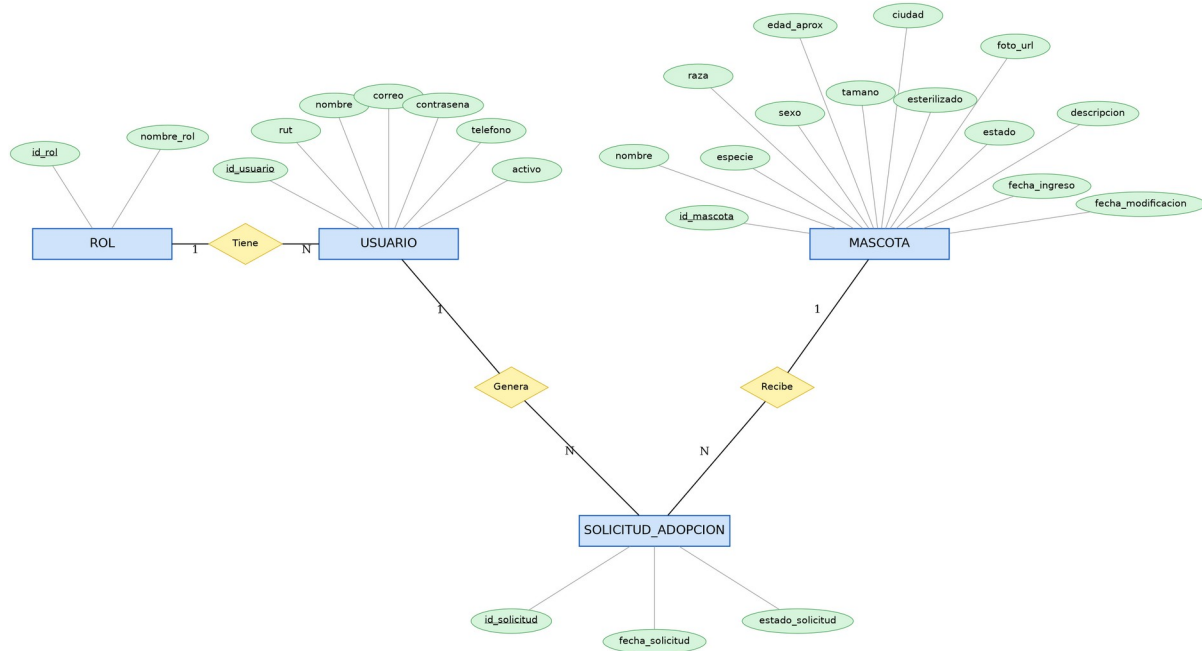


Figura 6: Modelo entidad-relación conceptual

Luego pasamos ese modelo a tablas. La Figura 7 muestra el modelo relacional, con las llaves primarias (PK) y foráneas (FK).

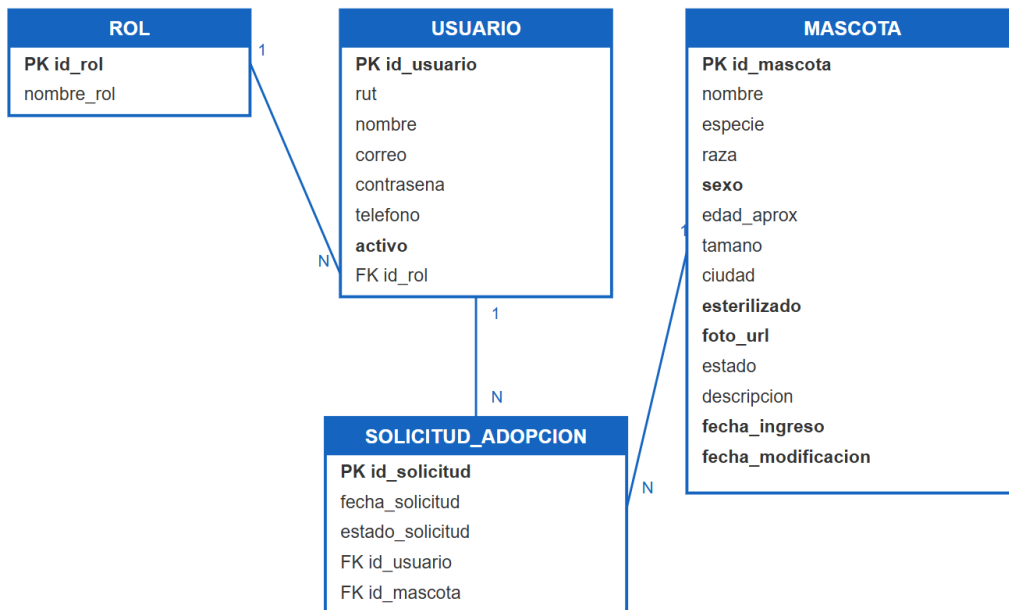


Figura 7: Modelo lógico relacional

Cardinalidades del modelo

- **ROL (1) — (N) USUARIO:** un rol puede estar asignado a muchos usuarios, pero cada usuario tiene un único rol.
- **USUARIO (1) — (N) SOLICITUD_ADOPCION:** un usuario puede generar muchas solicitudes a lo largo del tiempo.
- **MASCOTA (1) — (N) SOLICITUD_ADOPCION:** una mascota puede recibir muchas solicitudes, aunque solo una puede llegar a estado 'Aprobada'.

Esquema relacional (llave primaria subrayada, llave foránea en cursiva)

ROL (id_rol, nombre_rol)

USUARIO (id_usuario, rut, nombre, correo, contrasena, telefono, activo, *id_rol*)

MASCOTA (id_mascota, nombre, especie, raza, sexo, edad_aprox, tamano, ciudad, esterilizado, foto_url, estado, descripcion, fecha_ingreso, fecha_modificacion)

SOLICITUD_ADOPCION (id_solicitud, fecha_solicitud, estado_solicitud, *id_usuario*, *id_mascota*)

4.2 Diccionario de datos

Tabla ROL

Atributo	Tipo	Restricción	Descripción
id_rol	INT	PK, AUTO_INCREMENT	Identificador único del rol.
nombre_rol	VARCHAR(20)	UNIQUE, NOT NULL	Nombre del perfil de acceso: 'Usuario' o 'Administrador'.

Tabla USUARIO

Atributo	Tipo	Restricción	Descripción
id_usuario	INT	PK, AUTO_INCREMENT	Identificador único del usuario registrado.
rut	VARCHAR(12)	UNIQUE, NOT NULL	Rol Único Tributario, con formato validado.
nombre	VARCHAR(100)	NOT NULL	Nombre completo del usuario.
correo	VARCHAR(100)	UNIQUE, NOT NULL	Correo electrónico; se usa para iniciar sesión.
contrasena	VARCHAR(255)	NOT NULL	Contraseña transformada con bcrypt y salt (RNF03).
telefono	VARCHAR(15)	NULL	Teléfono móvil o fijo de contacto.
activo	TINYINT(1)	NOT NULL, DEFAULT 1	Estado de la cuenta (1: activa, 0: suspendida o inactiva).
id_rol	INT	FK, NOT NULL	Clave foránea hacia ROL; define qué puede hacer el usuario.

Tabla MASCOTA

Atributo	Tipo	Restricción	Descripción
id_mascota	INT	PK, AUTO_INCREMENT	Identificador único del animal rescatado.
nombre	VARCHAR(50)	NOT NULL	Nombre de la mascota.

Atributo	Tipo	Restricción	Descripción
especie	VARCHAR(20)	NOT NULL	Especie del animal (ej. 'Perro', 'Gato', 'Conejo'). Filtro opcional.
raza	VARCHAR(50)	NULL	Raza del animal, si se conoce, o mestizo.
sexo	ENUM('Macho', 'Hembra')	NOT NULL	Sexo del animal. Filtro opcional.
edad_aprox	INT	NULL	Edad estimada, en años.
tamano	VARCHAR(10)	NULL	Tamaño estimado: 'Pequeño', 'Mediano' o 'Grande'. Filtro opcional.
ciudad	VARCHAR(50)	NOT NULL	Ciudad donde está la mascota. Filtro obligatorio.
esterilizado	TINYINT(1)	NOT NULL, DEFAULT 0	Indica si está esterilizado: 1 (sí), 0 (no). Filtro opcional.
foto_url	VARCHAR(255)	NULL	Enlace a la foto principal del animal (RF02 y RF05).
estado	VARCHAR(20)	NOT NULL, DEFAULT 'Disponible'	Estado del proceso: 'Disponible' (filtro obligatorio) o 'Adoptada'.
descripcion	TEXT	NULL	Temperamento, personalidad y cuidados que necesita.
fecha_ingreso	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora en que se registró el animal (RF09).
fecha_modificacion	DATETIME	NOT NULL, ON UPDATE CURRENT_TIMESTAMP	Fecha y hora de la última edición de la ficha (RF09).

Tabla SOLICITUD_ADOPCION

Atributo	Tipo	Restricción	Descripción
id_solicitud	INT	PK, AUTO_INCREMENT	Identificador único de la postulación.
fecha_solicitud	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora exacta en que se envió la solicitud.
estado_solicitud	VARCHAR(20)	NOT NULL, DEFAULT 'Pendiente'	Estado del trámite: 'Pendiente', 'Aprobada' o 'Rechazada'.
id_usuario	INT	FK, NOT NULL	Clave foránea hacia USUARIO; identifica a quien postula.
id_mascota	INT	FK, NOT NULL	Clave foránea hacia MASCOTA; identifica al animal solicitado.

4.3 Integridad referencial

FK en USUARIO (id_rol) — ON DELETE RESTRICT: no se puede borrar un rol si todavía hay usuarios que lo tienen asignado. Así evitamos que quede un usuario sin rol, porque el sistema no sabría qué puede hacer.

FKs en SOLICITUD_ADOPCION (id_usuario, id_mascota) — ON DELETE RESTRICT: no se puede borrar un usuario ni una mascota que ya tenga solicitudes registradas. En vez de eliminarlos, los usuarios se desactivan (activo = 0) y las mascotas cambian de estado, y así se protege el historial de adopciones del refugio.

4.4 Supuestos y decisiones de diseño

- **Una sola solicitud aprobada por mascota:** una mascota puede recibir varias postulaciones, pero solo una puede quedar 'Aprobada'. Para asegurarlo, incluso si dos administradores aprueban al mismo tiempo, la aprobación se hace en una transacción SQL con bloqueo de fila (SELECT ... FOR UPDATE). En un solo paso se aprueba la solicitud elegida, la mascota pasa a 'Adoptada' y las demás solicitudes pendientes se rechazan.
- **La ciudad se guarda como texto simple:** en vez de crear una tabla aparte de ciudades, el atributo ciudad se guarda directo en MASCOTA y se elige desde una lista fija en el formulario. Lo decidimos así para que las búsquedas del catálogo sean más rápidas, ya que evita cruces de tablas (JOIN) innecesarios y ayuda a cumplir el tiempo del RNF01.
- **Un usuario puede postular varias veces:** una misma persona puede hacer distintas postulaciones a lo largo del tiempo (por ejemplo, por más de un animal), pero el sistema no le permite tener dos solicitudes 'Pendientes' por la misma mascota (RF10). Por eso la relación entre USUARIO y SOLICITUD_ADOPCION es de uno a muchos.

4.5 Consultas SQL implementadas

Estas son las 14 consultas SQL que respaldan el modelo de datos y el backend. Las consultas 1 a 4 (DDL) crean las tablas del modelo relacional y las consultas 5 a 7 (DML) cargan los datos iniciales. Las consultas 8 a 14 son las que ejecuta la API cuando se usa desde Swagger; entre ellas, las consultas 12, 13 y 14 forman la transacción de aprobación de una adopción, que se guarda completa o no se guarda (RNF02).

Nº	Categoría / Función	Tipo	Descripción de la consulta	Código SQL ejecutado
1	DDL - Tabla ROL	CREATE	Estructura para perfiles de acceso.	CREATE TABLE ROL (id_rol INT AUTO_INCREMENT PRIMARY KEY, ...);
2	DDL - Tabla USUARIO	CREATE	Estructura usuarios, clave foránea y borrado lógico.	CREATE TABLE USUARIO (id_usuario INT..., activo TINYINT DEFAULT 1...);
3	DDL - Tabla MASCOTA	CREATE	Catálogo de animales, filtros y auditoría de fechas.	CREATE TABLE MASCOTA (id_mascota INT..., estado VARCHAR DEFAULT 'Disponible'...);
4	DDL - SOLICITUD_ADOPCION	CREATE	Entidad relacional entre usuarios y mascotas.	CREATE TABLE SOLICITUD_ADOPCION (id_solicitud INT..., FK id_usuario, FK id_mascota);
5	DML - Carga Base ROL	INSERT	Inicialización de roles de acceso.	INSERT INTO ROL (nombre_rol) VALUES ('Administrador'), ('Usuario');
6	DML - Carga Base USUARIO	INSERT	Inserción de usuarios con claves encriptadas.	INSERT INTO USUARIO (rut, nombre, correo, contrasena...) VALUES (...);
7	DML - Carga Base MASCOTA	INSERT	Inserción del catálogo inicial de animales.	INSERT INTO MASCOTA (nombre, especie, raza, sexo...) VALUES ('Milo', 'Perro'...);
8	API - Búsqueda Catálogo	SELECT	Consulta con 2 filtros obligatorios + 4 opcionales.	SELECT * FROM MASCOTA WHERE estado='Disponible' AND ciudad='Valparaíso'...;
9	API - Registro Usuarios	INSERT	Alta de nuevo adoptante desde la API (POST).	INSERT INTO USUARIO (rut, nombre, correo, contrasena...) VALUES (...);
10	API - Borrado Lógico	UPDATE	Desactivación de cuenta de usuario (activo = 0).	UPDATE USUARIO SET activo = 0 WHERE id_usuario = 2;
11	API - Crear Postulación	INSERT	Registro de nueva solicitud de adopción.	INSERT INTO SOLICITUD_ADOPCION (id_usuario, id_mascota) VALUES (2, 1);
12	API - Transacción (Aprobar)	UPDATE	Cambio de estado de la solicitud a 'Aprobada'.	UPDATE SOLICITUD_ADOPCION SET estado_solicitud = 'Aprobada' WHERE id_solicitud = 1;
13	API - Transacción (Mascota)	UPDATE	Cambio de estado de la mascota a 'Adoptada'.	UPDATE MASCOTA SET estado = 'Adoptada' WHERE id_mascota = 1;
14	API - Transacción (Rechazar)	UPDATE	Rechazo automático de solicitudes duplicadas.	UPDATE SOLICITUD_ADOPCION SET estado_solicitud = 'Rechazada' WHERE id_mascota = 1 AND id_solicitud != 1 AND estado_solicitud = 'Pendiente';

5. Actualizaciones respecto a la propuesta enviada

Este cuadro resume qué cambiamos con respecto a la propuesta inicial y por qué.

Sección del informe	Modificación realizada	Justificación / observación atendida
1. Introducción y problema	Reescribimos el contexto y el problema con un lenguaje más directo y fácil de entender.	Explicar con claridad cómo funcionan los refugios y cómo afecta la situación a los animales.
1.2 Tecnologías utilizadas	Agregamos la sección con el detalle de las tecnologías: Node.js, Express.js, MySQL, mysql2, Swagger UI, dotenv y CORS.	Faltaba describir las tecnologías del proyecto, que es un punto exigido en la rúbrica.
2.1 Requerimientos funcionales	Simplificamos la redacción de RF01 a RF10. RF03 ahora define 2 filtros obligatorios (Estado y Ciudad) y 4 opcionales (Especie, Tamaño, Sexo y Esterilizado).	Que los requerimientos coincidan con los prototipos y con los diagramas.
2.1 Requerimientos no funcionales	Ajustamos RNF01 a RNF10; corregimos RNF06 (formato JSON) y aclaramos la capa Modelo del MVC (RNF08) y la transmisión de datos (RNF09).	Corregir imprecisiones sobre la arquitectura MVC, el formato de datos y la seguridad.
2.2 y 2.3 Casos de uso	Separamos la gestión del administrador en dos casos de uso: Mantenedor de mascotas y Mantenedor de usuarios. Agregamos las etiquetas «include» al diagrama.	Tener al menos dos mantenedores tipo CRUD y mostrar bien qué casos requieren sesión.
2.4 Diagramas de secuencia	Rehicimos el diagrama del catálogo para usar los mismos filtros que RF03 (obligatorios y opcionales).	Mantener la coherencia entre requerimientos, diagramas y prototipos.
3. Prototipos	Actualizamos la pantalla del catálogo con Estado y Ciudad como filtros obligatorios, y Especie, Tamaño, Sexo y Esterilizado como opcionales.	Que las pantallas sean fieles a lo que se especificó.
4.1 Modelo ER y relacional	Completamos el modelo ER con todos los atributos y actualizamos el esquema relacional para incluir todos los campos.	Que el modelo, las tablas y el diccionario de datos coincidan entre sí.
4.2 y 4.4 Modelo de datos	Agregamos el campo activo en USUARIO y los campos sexo, esterilizado, foto_url, fecha_ingreso y fecha_modificacion en MASCOTA.	Coherencia con el catálogo, las fotos, el registro de fechas y la desactivación de cuentas.
4.3 y 4.4 Integridad y supuestos	Detallamos cómo se asegura una sola adopción aprobada con transacciones SQL y bloqueo de fila (FOR UPDATE).	Atender la observación técnica del profesor sobre solicitudes simultáneas.
4.5 Consultas SQL	Agregamos la sección con las 14 consultas SQL (DDL, DML y API) de la implementación.	Mostrar las consultas que respaldan el modelo de datos y el backend.